

CT313H: Web Technologies and Services

Introduction

Bùi Võ Quốc Bảo | Department of IT

Cần Thơ, 2025

Course overview

In this course, you will learn:

- About the important concepts in modern web development: different ways of rendering HTML pages, server-side JavaScript, advanced concepts in JavaScript (event loop, closure, promise / async-await, JSDoc, ...)
- About the concepts of web service, REST API, authentication schemes for API
- To build and deploy secure REST-like APIs for CRUD (create/read/update/delete) operations with Node.js and Knex.js, to document the API with OpenAPI spec
- To build and deploy SPAs with Vue.js

Prerequisites

Before starting this course it is expected you will already have:

- Basic understanding of web development (PHP, HTML, CSS, Javascript)
- Basic skills in programming, data structures
- Knowledge of SQL and database management systems

Course outcomes

At the end of this course, participants will be able to:

- CO1: Explain the different ways of rendering HTML pages and how SPA works
- CO2: Explain REST and the different authentication schemes for REST APIs
- CO3: Build and deploy secure REST-like APIs for CRUD (create/read/update/delete) operations with Node.js and Knex.js
- CO4: Document REST APIs using OpenAPI specification
- CO5: Build and deploy SPAs (Single Page Application) with Vue.js using standard libraries and recommended practices

Teaching channels

- Physical Classroom
- Google Classroom
- GitHub Classroom
- [CTU Elearning](#)
- Email: bvqbao@ctu.edu.vn

Content

Chapter 1: Bootstrap

Chapter 2: Node.js, Express

Chapter 3: HTTP APIs, OpenAPI

Chapter 4: JavaScript + JSDoc

Chapter 5: Knex.js and PostgreSQL

Chapter 6: SPAs with Vue.js

Chapter 7: Deployment

Labs

Lab 1-2: Contact Management App – Backend (API, Database)

Lab 3-4: Contact Management App – Frontend (SPA)

Schedule (subject to change)

- W01: Introduction, Bootstrap
- W01: Node.js & Express
- W02: Node.js & Express
- W02: HTTP APIs / OpenAPI
- W03: JavaScript + JSDoc, Knex.js
- W03-04: Lab 1-2 (Tutorials)
- W05: Vue.js, Single File Component (SFC)
- W05: Vue Router
- W06: State Management
- W06: Deployment
- W07-08: Lab 3-4 (Tutorials)
- W09-11: Project
- W12: Project Presentations
- W13: Exam

References

- JavaScript + JSDoc
 - <https://javascript.info/>
 - <https://eloquentjavascript.net/index.html>
 - <https://jsdoc.app/>

References

- Node.js and Express
 - <https://nodeschool.io/vi/>
 - <https://nodejs.dev/learn>
 - <https://www.w3schools.com/nodejs/>
 - <https://expressjs.com/>
- Knex.js
 - <https://knexjs.org/guide/>

References

- Vue.js

- <https://vuejs.org/guide/introduction.html>
- <https://router.vuejs.org/guide/>
- <https://tanstack.com/query/latest/docs/framework/vue/overview>
- <https://pinia.vuejs.org/core-concepts/>

- Bootstrap

- [http://getbootstrap.com\(.vn\)](http://getbootstrap.com(.vn))
- <http://w3schools.com/bootstrap>
- <http://bootswatch.com>

References

- REST API
 - http://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
 - REST API Tutorial (<http://www.restapitutorial.com/>)
 - The RESTful Cookbook (<http://restcookbook.com/>)
 - Swagger Documentation (<https://swagger.io/docs/specification/about/>)
- Source code management with Git
 - <http://git-scm.com/>
 - <http://rogerdudler.github.io/git-guide/>
 - <http://schacon.github.io/git/gittutorial.html>

Evaluation

	Weight	CO1	CO2	CO3	CO4	CO5
4 Labs / Tutorials	30%			X	X	X
Group Project	40%			X	X	X
MCQ	30%	X	X	X	X	X

- Please note that:
 - No lab submission = No final exam = F
 - Late submission penalty: -0.5 points / week

Project

- Build an SPA web application
 - Project plan:
 - Week 01-02: Create groups of 2 students and determine a project topic for each group
 - Week 03-06: Design, implement, and document REST API
 - End of week 06: Submit the report in progress
 - Week 07-11: Design and implement SPA
 - End of week 11: Submit the final report
 - Week 12: Project presentations

Project

- Build an SPA web application
 - Students are free to suggest project ideas, for example:
 - Book/cafe/film/... review app
 - Shopping / management / utility app
 - ...
 - Tech stack: **Node.js/Express, Knex.js, PostgreSQL, Vue.js**

Project

- Build a SPA web application

Evaluation components \ Score*	1 (0%)	2 (25%)	3 (50%)	4 (75%)	5 (100%)
Design and implement REST API for CRUD operations	0	0,5	1	1,5	2
Document REST API with OpenAPI (Swagger)	0	0,5	1	1.5	2
Protect/secure REST API with user authentication and rate limiting	0	0,5		1	
Design and build web application pages from Vue.js components and Vue Router	0	0,5	1	1,5	2
Apply appropriately state management solutions (composables, provide/inject, Pinia or TanStack Query)	0	0,5	1	1,5	2
Deploy REST API and SPA	0	0,5		1	

* Students get scores based on the final product and the student's understanding of the source code (via the submitted report, QA session)

Git and GitHub

- The source code (labs, project) must be managed by git and uploaded to GitHub
 - Each student must have an account on [GitHub](#)
 - Each project group member must contribute code to the repo
- Git/GitHub docs are provided by the instructor



Rendering HTML pages

There are mainly 3 strategies for architecting a web application:

1. Server-Side Rendering (SSR)

Server sends a new HTML page for each unique path

2. Single-Page Application (SPA)

Server sends the exact same web page for every unique path (and the page runs JS to change what it look like)

3. Combination of 1 and 2 ("Isomorphic" / "Universal")

Preload data using server-side rendering before running the application on the client-side

Rendering HTML pages

There are mainly 3 strategies for architecting a web application:

1. Server-Side Rendering (SSR)

Server sends a new HTML page for each unique path

- Also called Multi-Page Application (MPA)
- Some call it [Hypermedia-Driven Application](#)
- Dynamic websites in traditional senses
- Suitable for public-facing sites

Rendering HTML pages

There are mainly 3 strategies for architecting a web application:

1. Server-Side Rendering (SSR)

Server sends a new HTML page for each unique path

2. Single-Page Application (SPA)

Server sends the exact same web page for every unique path (and the page runs JS to change what it look like)

3. Combination of 1 and 2 ("Isomorphic" / "Universal")

Preload data using server-side rendering before running the application on the client-side

Rendering HTML pages

There are mainly 3 strategies for architecting a web application:

1. Server-Side Rendering (SSR)

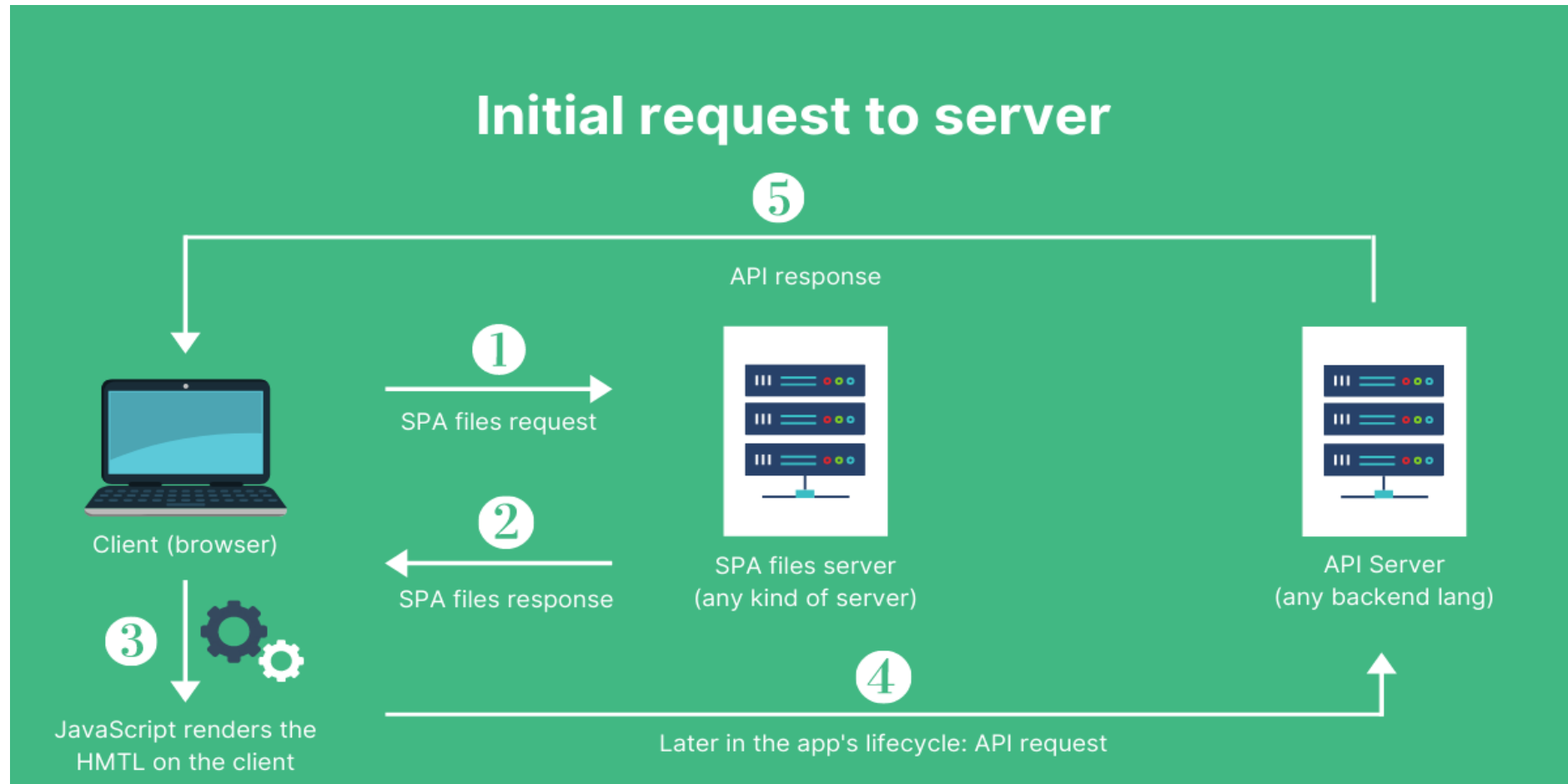
Server sends a new HTML page for each unique path

2. Single-Page Application (SPA)

Server sends the exact same web page for every unique path (and the page runs JS to change what it look like)

- Also known as Client-Side Rendering (CSR)
- Usually called web applications instead of websites
- Suitable for internal / management web apps

Single-Page Application (SPA)



Rendering HTML pages

There are mainly 3 strategies for architecting a web application:

1. Server-Side Rendering (SSR)

Server sends a new HTML page for each unique path

2. Single-Page Application (SPA)

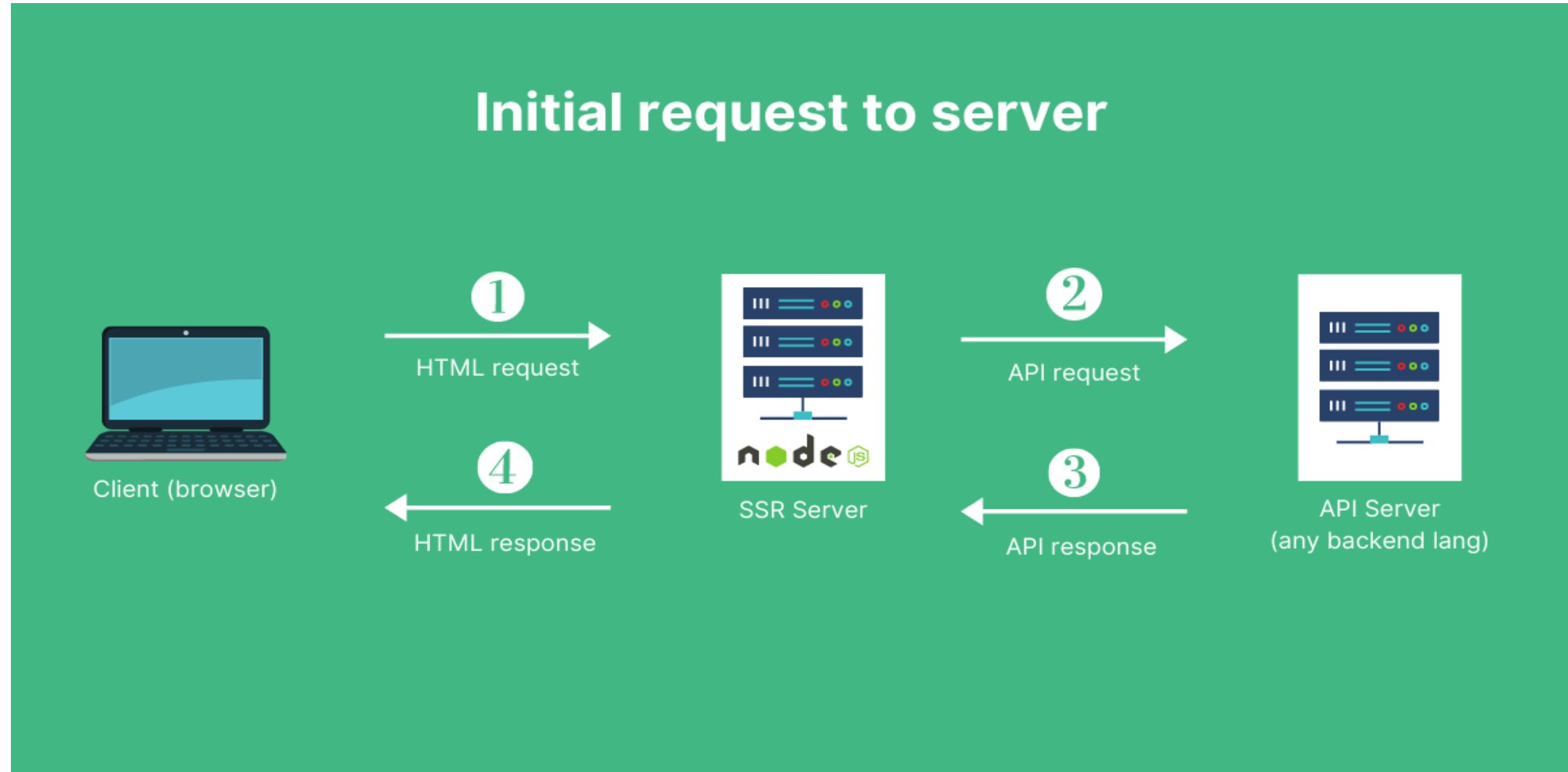
Server sends the exact same web page for every unique path (and the page runs JS to change what it look like)

3. Combination of 1 and 2 ("Isomorphic" / "Universal")

Preload data using server-side rendering before running the application on the client-side

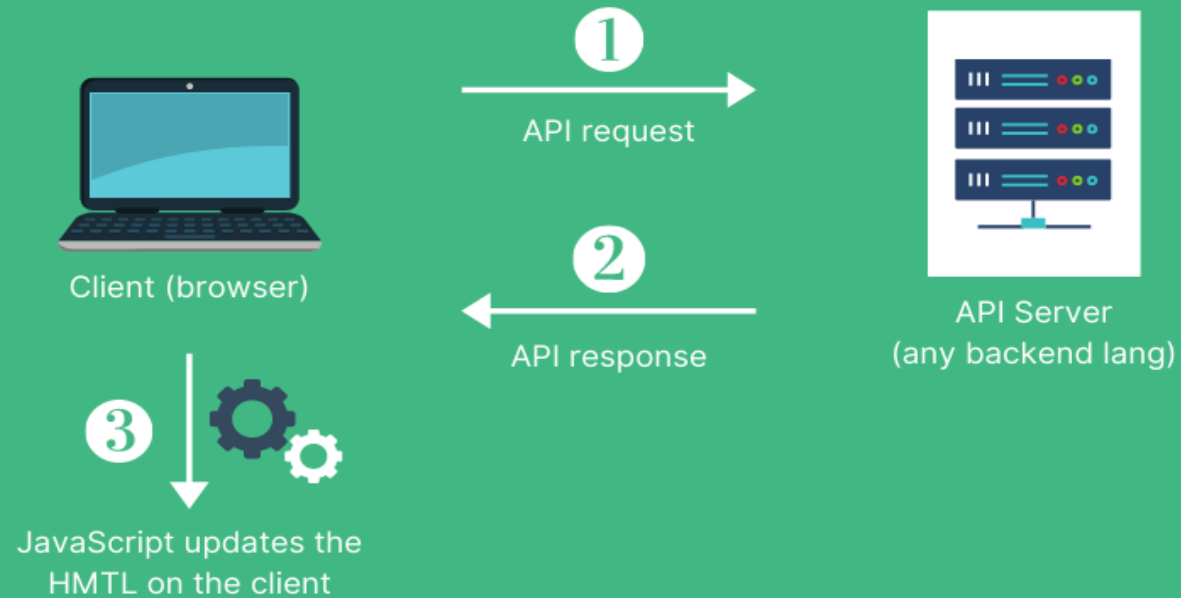
Popular frameworks: Next (React), Nuxt (Vue), (hosted) Blazor WebAssembly, Laravel + Inertiajs + Vue/React, Laravel + Livewire + Alpinejs, ...

SSR + SPA (Universal)



SSR + SPA (Universal)

**All following requests:
it goes directly from the SPA to the API server**



Rendering HTML pages

1. Server-Side Rendering (SSR)

- Very fast startup
- User interactions are slow
- Work well with search engines

2. Single-Page Application (SPA)

- Initial page load is low
- No round trips to server

3. Isomorphic / Universal Rendering

Combine advantages of SSR and SPA

Questions?